

CYBERSHIELD

Software Requirements Specification

An Evidence-Driven URL Phishing Detection Platform

LDRP Institute of Technology and Research

Academic Project Report

Submitted by: Neel Savsani

Project Guide: _____

Date: 06 August 2026

Acknowledgement

This Software Requirements Specification has been prepared for the CyberShield academic project. The report documents the requirements and implementation completed to date. We acknowledge the guidance of faculty members, project reviewers, peers, and the open-source communities whose tools support the design, implementation, testing, and documentation of the system.

Abstract

CyberShield is a web-based URL phishing analysis platform that investigates a submitted public HTTP or HTTPS address using live infrastructure and page-behaviour evidence. The system performs URL safety checks, HTTP, DNS, domain, TLS, browser, HTML, JavaScript, form, and optional reputation analysis. It produces a risk probability, a user-facing verdict, contributing indicators, a feature breakdown, and a scan-specific screenshot when capture succeeds. The current classifier is an explainable baseline intended for decision support; it is not a validated automated blocking model. This document specifies the system using the IEEE 830-oriented SRS structure and records the implemented functionality, constraints, data handling, interfaces, and planned extensions.

Table of Contents

1. Introduction
2. Overall Description
3. Specific Requirements
4. System Models
5. Appendices

1. Introduction

1.1 Purpose

CyberShield assists users in assessing the phishing risk of a URL before they interact with it. The system collects evidence from multiple sources rather than relying only on URL spelling, brand keywords, or fixed character-count rules. It is designed to explain the observed evidence so that the user can make an informed decision.

1.2 Scope

The implemented scope is public URL analysis. A user can submit an HTTP or HTTPS URL from the dashboard, receive an analysis result, save it to personal history, reopen it from History or Reports, and print the result as a PDF. Email/text analysis and image upload are visible interface options but are not currently implemented as analysis workflows.

1.3 Definitions, Acronyms and Abbreviations

Table 1: Definitions and Abbreviations

Term	Definition
SRS	Software Requirements Specification.
SSRF	Server-Side Request Forgery; a risk in which a submitted URL attempts to reach internal services.
TLS	Transport Layer Security, used by HTTPS.
DNS	Domain Name System.
Evidence-v1	CyberShield feature schema derived from collected evidence.

1.4 References

- ISO/IEC/IEEE 29148 - Requirements engineering.
- FastAPI, Playwright Python, Firebase, and OWASP SSRF Prevention documentation.
- CyberShield source code, API models, frontend scripts, and current project README.

1.5 Document Overview

Section 2 describes the product and operating context. Section 3 specifies functional, non-functional, interface, and performance requirements. Section 4 provides logical system models. Section 5 includes traceability and implementation-status appendices.

2. Overall Description

2.1 Product Perspective

CyberShield is a multi-page web application backed by a FastAPI service. The frontend provides dashboard, authentication, history, reports, profile, settings, and result views. The backend validates URLs, coordinates collectors, converts evidence into stable features, and returns an explainable classification. Firebase Authentication and Firestore support the frontend account and saved-analysis experience, while SQLite-backed platform endpoints are available for local API workflows.

2.2 Product Functions

- Accept and normalize a public HTTP/HTTPS URL.
- Block localhost and private or non-public network destinations before collection.
- Collect HTTP, DNS, domain, TLS, browser, HTML, JavaScript, form, and optional reputation evidence.
- Capture a screenshot from the controlled browser session when available.
- Calculate an explainable risk score and show indicators, features, and user guidance.
- Store and reopen a registered user's complete analysis record.

2.3 User Classes and Characteristics

Table 2: User Classes

User Class	Characteristics
Guest	Can submit URLs and use a temporary dashboard history; account-only history, reports, profile, and settings are restricted.
Registered user	Uses Firebase Authentication and can retain, view, filter, delete, print, and revisit personal saved analyses.
Administrator/developer	Runs the API, maintains configuration, configures optional reputation providers, and monitors the project environment.

2.4 Operating Environment

The frontend runs in modern desktop or mobile browsers. The backend runs in a Python environment supporting FastAPI and Playwright Chromium. Network access is required for public targets, DNS and TLS queries, optional threat-intelligence providers, and Firebase cloud services. On Windows, browser-capable analysis uses a Proactor event loop in a worker thread to support Playwright subprocess creation.

2.5 Constraints, Assumptions and Dependencies

The system depends on public network availability, target-site behaviour, browser compatibility, and external services. A missing provider key or failed collector must be represented as unavailable evidence, not as a false safe or malicious result. The risk score is advisory: it must not be used as an automatic production blocking decision until a versioned model has been trained and validated on labelled data.

3. Specific Requirements

3.1 Functional Requirements

Table 3: Functional Requirements

ID	Requirement
FR-01	The system shall provide registration, sign-in, password recovery, sign-out, and guest entry.
FR-02	The system shall accept complete HTTP/HTTPS URLs and reject malformed, local, private, link-local, reserved, or otherwise non-public targets before analysis.
FR-03	The system shall collect HTTP reachability/redirect, DNS, domain age, TLS certificate, and configured reputation evidence when available.
FR-04	The system shall render an admitted URL in a controlled browser context and collect final URL, title, browser events, page content, forms, and a scan-specific screenshot when possible.
FR-05	The system shall derive evidence-v1 features and return a phishing probability, risk level, ranked contributors, and model status notice.
FR-06	The Result page shall show score, verdict, input, timestamp, recommendation, indicators, feature values, and the matching screenshot.
FR-07	Registered users' saved analysis records shall retain content, score, verdict, indicators, features, screenshot URL, and timestamps.
FR-08	History and Reports shall reconstruct the same complete result data as the initial analysis result when a user selects View.
FR-09	The system shall allow a saved result to be printed or saved as a PDF through the browser print flow.

3.2 Non-functional Requirements

Table 4: Non-functional Requirements

ID	Category	Requirement
NFR-01	Security	Reject unsafe destinations before any HTTP, TLS, or browser collection to reduce SSRF risk.

ID	Category	Requirement
NFR-02	Privacy	A screenshot must belong to the particular analysis that generated it; the Result page must not fall back to a shared latest screenshot.
NFR-03	Reliability	Collector failures shall return partial labelled evidence and shall not prevent browser resources from closing.
NFR-04	Usability	The interface shall explain risk using a concise verdict, contributors, recommendation, and accessible feature values.
NFR-05	Maintainability	Collectors, feature extraction, classification, and frontend rendering shall remain separated components.
NFR-06	Ethical use	The baseline/synthetic model limitation shall be documented before any automated blocking is considered.

3.3 External Interface Requirements

3.3.1 User Interface

The Dashboard shall provide URL input, progress, summary statistics, and recent scans. The Result page shall display complete result evidence. History and Reports shall allow a user to find and reopen saved analyses. Profile and Settings are account-only screens.

3.3.2 API Interface

Table 5: API Interfaces

Endpoint	Method	Purpose
/analyze	POST	Returns the synchronous evidence-driven URL analysis response.
/api/scans	POST/GET	Submits asynchronous platform scans and retrieves local platform scan records.
/api/scans/{scan_id}	GET	Retrieves a specific queued scan after access checks.
/health	GET	Reports that the backend service is online.
/reports	Static	Serves scan-specific browser screenshot files.

3.4 Performance Requirements

The system shall promptly reject invalid or unsafe targets. For admitted URLs, execution time is dependent on live web, DNS, TLS, browser, and provider response conditions; therefore the interface shall show progress while analysis is active. Browser analysis shall execute away from the server's main event loop where the platform requires it. External network calls shall use bounded timeouts and failures shall produce a readable result state.

3.5 Design Constraints

CyberShield currently supports only publicly reachable HTTP/HTTPS URLs. Optional VirusTotal and Google Safe Browsing checks require user-supplied server-side API keys. OpenPhish and URLhaus are declared future feed integrations. The deployed production environment must apply restrictive CORS, secret management, HTTPS, Firebase Security Rules, rate limiting, and retained-data controls.

4. System Models

4.1 Use Case Model

The main actors are guest users, registered users, and administrators. The use case model represents the implemented user workflow and the administrative operational role.

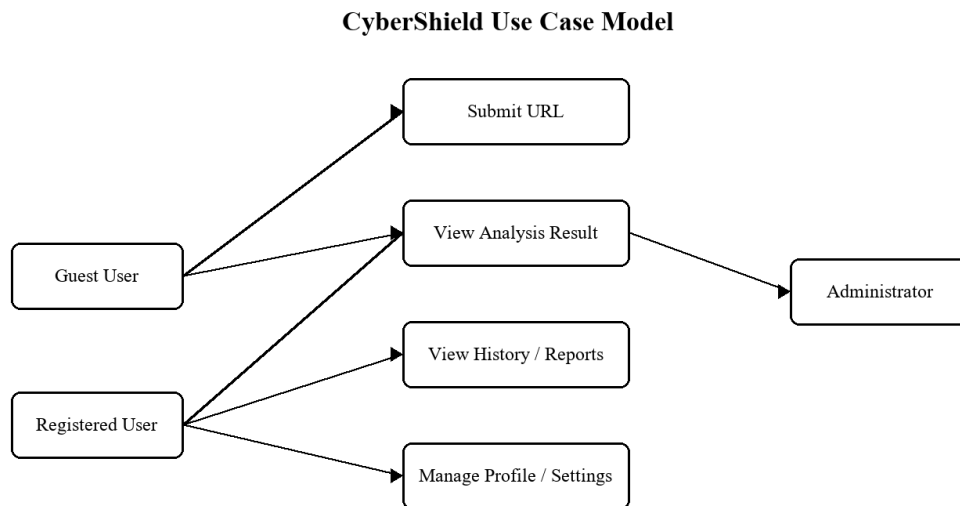


Figure 1: CyberShield Use Case Model

4.2 Data Flow Diagram

At level 0, the user submits a URL to CyberShield. The platform exchanges requests with public web/DNS/TLS services and optional reputation providers, then stores the user-visible saved result in Firestore for authenticated users.

CyberShield Level-0 Data Flow Diagram

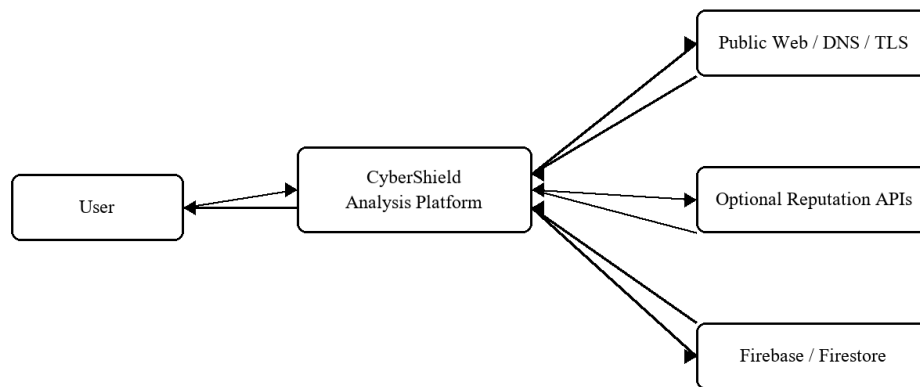


Figure 2: CyberShield Level-0 Data Flow Diagram

4.3 Logical Data Model

The application maintains user-owned analyses. Each analysis retains the submitted content, verdict, risk score, timestamp, indicators, extracted features, and screenshot URL. User settings are associated with the same authenticated user scope.

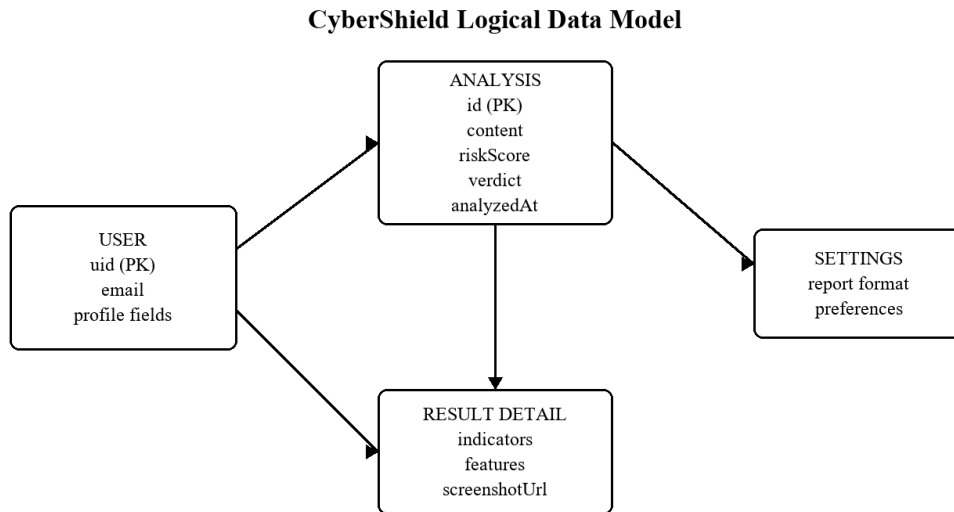


Figure 3: CyberShield Logical Data Model

4.4 Analysis Sequence

- Normalize and validate the URL.
- Resolve hostname and reject unsafe/non-public targets.
- Collect infrastructure and reputation evidence.
- Render the page in Playwright and inspect page, script, and form evidence.
- Extract evidence-v1 features and predict risk with contributor explanations.
- Save the complete user-visible record and render the result.

5. Appendices

Appendix A - Requirement Traceability Matrix

Table 6: Requirement Traceability Matrix

Requirement	Implementation evidence	Validation
FR-02	URL safety gate and orchestrator	Submit malformed, localhost, and private targets; confirm rejection before collection.
FR-03/FR-04	Analyzer modules and Playwright browser session	Analyze an admitted public target and inspect returned evidence/screenshot.
FR-05/FR-06	Feature extractor, classifier, result.js	Verify probability, contributors, indicators, features, and verdict display.

Requirement	Implementation evidence	Validation
FR-07/FR-08	Firebase data layer, history.js, reports.js	Save a result and reopen it from History and Reports; compare detail with the fresh result.
NFR-02	Per-analysis screenshot URL mapping	Confirm a saved result shows only its own screenshot or hides the panel when absent.

Appendix B - Current Implementation Status

Table 7: Current Implementation Status

Area	Status	Notes
URL analysis	Implemented	Public URL analysis via live collector pipeline.
Browser screenshot	Implemented	Stored and restored as part of the matching analysis record.
History/Reports result fidelity	Implemented	Features and screenshot URL are passed into the Result page.
Email/text and image analysis	Planned	Interface placeholders exist; analysis workflow is not implemented.
OpenPhish/URLhaus integration	Planned	Provider placeholders are present; production integration remains future work.
Validated production model	Planned	Baseline/demonstration model must be replaced by a versioned validated model.